

Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering, Pune

Agentic AI-Based Knowledge Graph Assistant for Engineering Teams

Nexus — Detailed Project Report

By

Mehul Rawat (123B1C063)

Payal Lokhande (123B1C017)

Subject: Agentic AI | Formative Assessment

Academic Year 2026–2027

Course Coordinator - Dr. Santwana Gudadhe

Table of Contents

Abstract

1. Introduction
    1.1 Background
    1.2 Need for Agentic AI
    1.3 Project Scope
2. Problem Statement
3. Objectives
4. Proposed Agent Architecture
    4.1 System Architecture Diagram
    4.2 Component Responsibilities
    4.3 Technology Stack
    4.4 Module Inventory
5. Prompt Design
6. RAG Implementation
7. Tool/API Integration
8. Agent Workflow
9. Safety, Ethics and Responsible AI
10. Results and Test Report
11. Limitations
12. Future Scope
13. Conclusion
14. References and Tools Used

Appendix A: How to Run

Appendix B: Project Structure

Abstract

Nexus is an Agentic AI application that answers engineering questions about an organisation — who owns a service, what depends on what, which standards apply, and what the live source code does — by planning, retrieving and reasoning over a company knowledge graph and GitHub repositories. Organisational documents are converted into a knowledge graph of entities and sourced facts, each tagged as verified or proposed. A single agent built on Claude Sonnet 5.5 (Anthropic API, tool use with adaptive thinking) decides at every step which of nine read-only tools to call: hybrid retrieval (SQLite full-text search combined with local embeddings), entity lookup, multi-hop relationship traversal, a review queue and four GitHub REST API tools. It iterates until it can answer, and every reasoning step and tool call is streamed live to a web page where the entities it used light up on an interactive graph. The same tools are exposed through a Model Context Protocol (MCP) server so that external AI clients such as Claude Code can use them. The prototype is implemented in Python (Flask, SQLite, fastembed) with a demonstration knowledge base of 28 documents for a fictional company. Eight functional test cases were executed and passed.

1. Introduction

1.1 Background

In software organisations, knowledge about systems is spread across design documents, team pages, engineering standards, runbooks and code repositories. A new engineer or an on-call responder must read several of these sources and connect them by hand to answer even simple questions such as “who owns the billing service?” or “what stops working if the database goes down?”.

1.2 Need for Agentic AI

A chatbot answers from what it already knows and cannot see the organisation’s own data, so it guesses. These questions need several dependent operations: find the right entity, read its facts, follow its dependencies across multiple hops, and sometimes inspect live code. The order of those operations depends on what each step returns. Nexus therefore uses an LLM agent that chooses its next tool after every observation — genuine autonomous planning and tool selection rather than a fixed pipeline — and grounds its answer in retrieved, sourced facts.

1.3 Project Scope

The application covers document ingestion into a knowledge graph, hybrid retrieval, an autonomous question-answering agent with a live execution trace, human review of unverified facts, read-only GitHub integration and an MCP server. It runs locally for classroom demonstration with a fictional company dataset (Orbit Labs) and is not a production service.

2. Problem Statement

- Scattered knowledge: ownership, dependencies and standards live in different documents and repositories.
- Multi-hop questions: impact and ownership questions need chains of lookups, not a single search.
- Untrustworthy answers: a plain LLM may invent facts and cannot show where an answer came from.
- Unverified information: documents contain uncertain claims that should not be treated as fact.

- Opaque reasoning: users cannot see how an AI system reached its conclusion.

Traditional approach	Nexus approach
Read many documents manually	Agent retrieves and connects the relevant entities automatically
Keyword search only	Hybrid search by keyword and by meaning (embeddings)
Follow dependencies by hand	Multi-hop graph traversal tool
Generic chatbot answers	Answers grounded in sourced facts, with unverified facts flagged
Opaque AI output	Every reasoning step and tool call shown live; used entities highlighted

3. Objectives

- Convert organisational documents into a knowledge graph of entities and sourced, status-tagged facts.
- Implement a RAG layer combining keyword (full-text) and semantic (embedding) retrieval.
- Build an autonomous agent that plans and calls tools in a loop until it can answer.
- Integrate an external API (GitHub REST) for questions about live source code.
- Expose the same tools through the Model Context Protocol for other AI clients.
- Make the agent's reasoning observable and keep it read-only, with human approval for data changes.
- Validate the behaviour with documented test cases.

4. Proposed Agent Architecture

4.1 System Architecture Diagram

Figure 1 shows the architecture. The user's question reaches the agent loop through the Flask server; the agent repeatedly asks Claude which tool to use, runs it through the tool router against the knowledge graph or GitHub, and streams every step back to the page. The MCP server exposes the same tools to external clients.

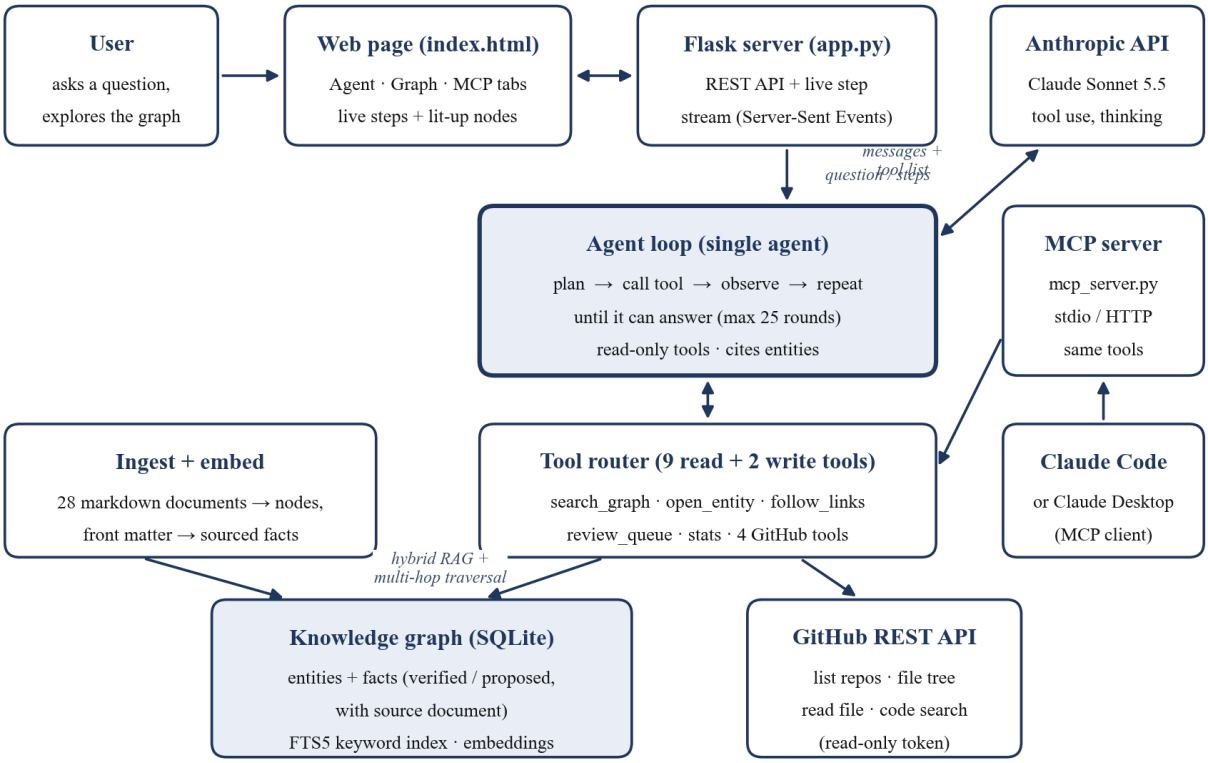


Figure 1: Nexus system architecture

Layer	Components / information flow
1. Users	Engineer, on-call responder, new joiner, reviewer
↓ Presentation	Single web page: Agent, Graph and MCP tabs; live step list; interactive graph
↓ Application API	Flask routes; Server-Sent Events stream for live agent steps
↓ Reasoning	Agent loop on Claude Sonnet 5.5 with tool use and adaptive thinking
↓ Tools	Graph search, entity, traversal, review queue, statistics, 4 GitHub tools
↓ Knowledge	SQLite knowledge graph, FTS5 keyword index, embedding vectors
↓ Results and controls	Cited answer, execution trace, highlighted entities, human approval of facts

Figure 2: Layered view of the architecture

4.2 Component Responsibilities

Component	Main responsibility
Agent loop	Plans the next step, calls tools, reads results, decides when it can answer (max 25 rounds).
Tool router	Maps a tool name chosen by the model to a Python function and returns its result as text.
Knowledge graph	Stores entities and facts with status (verified / proposed / rejected) and source.
Retriever	Hybrid keyword + embedding search merged by reciprocal rank fusion.
Graph traversal	Follows typed relationships up to 4 hops (e.g. dependency blast radius).
GitHub connector	Lists repositories, file trees, reads files and searches code (read-only).
MCP server	Exposes all tools to Claude Code / Claude Desktop over stdio or HTTP.
Web interface	Shows each step live, lights up used entities, supports review of proposed facts.

4.3 Technology Stack

Layer	Technology
Language	Python 3.13
Reasoning model	Claude Sonnet 5.5 via Anthropic Python SDK (tool use, adaptive thinking)
Web server	Flask 3.1 with Server-Sent Events
Storage	SQLite (knowledge graph + FTS5 full-text index)
Embeddings	fastembed 0.9 with BAAI/bge-small-en-v1.5 (384 dimensions, runs locally)
External API	GitHub REST API via httpx (read-only personal access token)
Interoperability	Model Context Protocol Python SDK 2.3 (stdio and streamable HTTP)
Frontend	Single HTML page with JavaScript, vis-network (graph), marked + DOMPurify (answers)

4.4 Module Inventory

Module (file / section)	Contents
app.py §1 Setup	Settings, .env loading, database tables
app.py §2 Graph	Entity lookup, multi-hop traversal, add fact, approve/reject, statistics
app.py §3 Ingest	Markdown documents → entities and sourced facts
app.py §4 Search	Keyword search, embeddings, hybrid retrieval
app.py §5 Tools	Nine read tools, two write tools, tool schemas, tool router
app.py §6 Agent	System prompt and the agent loop (streams each step)
app.py §7 Web server	Flask routes for the page, graph, search, review and agent stream
mcp_server.py	MCP server registering the same tool functions
static/index.html	Agent, Graph and MCP tabs

5. Prompt Design

Prompt engineering in Nexus works at three levels: a system prompt that defines role, domain vocabulary and rules; tool descriptions that teach the model when to use each tool; and model settings that control how much it reasons and expose that reasoning to the user.

Prompt element	Purpose / guardrail
Role definition	Assistant for a company knowledge graph; defines entity types.
Domain vocabulary	Explains each relationship: ownedBy, dependsOn, memberOf, ledBy, mustSatisfy, maintainedBy, relatesTo.
Trust rules	Distinguishes verified from proposed facts; must say when a proposed fact is used.
Tool-selection policy	Search first, open the entity before claiming something is missing, GitHub only for live code.
Grounding rules	Never invent facts; name the entities used; answer in Markdown.
Tool descriptions	Each tool’s docstring is sent as its description, with an input schema from its type hints.
Adaptive thinking	Model decides how much to reason; readable summaries are shown to the user.

5.1 System Prompt (excerpt)

"You are Nexus, an assistant for a company knowledge graph. It holds the company's services, teams, people, standards, tools and guides, linked by facts (ownedBy, dependsOn, memberOf, ledBy, mustSatisfy, maintainedBy, relatesTo). Facts are 'verified' (trusted) or 'proposed' (unconfirmed — say so when you use one). Use search_graph to find entities, open_entity to read one in full, and follow_links to trace dependencies or ownership. Only use the GitHub tools when the answer needs live source code. Never say a fact is missing before opening the entity. Answer in Markdown and name the entities you used. Never invent facts."

5.2 Validation Strategy

- Tool inputs follow JSON schemas generated from each function's type hints.
- Only registered tools can run; an unknown tool name returns an error message instead of executing anything.
- Tool errors are returned to the model as text so it can re-plan instead of crashing.
- Tool results are truncated (20,000 characters) to keep context bounded.
- Refusals from the model's safety classifiers are detected and answered politely.

6. RAG Implementation

6.1 Knowledge Base

The knowledge base contains 28 markdown documents for a fictional company: 7 services, 3 teams, 7 people, 4 engineering standards, 4 tools and 3 guides. Each document becomes an entity. Its front matter defines relationships, e.g. "dependsOn: postgres, redis", which become facts tagged with the source document. A leading "?" marks a fact as proposed (unverified). The current graph holds 28 entities and 75 facts (73 verified, 2 proposed).

6.2 Indexing

- Keyword index: SQLite FTS5 with the Porter stemmer over each entity's name, description and text.
- Semantic index: each entity is embedded with BAAI/bge-small-en-v1.5 (384-dimensional, normalised) using fastembed, stored as a vector in SQLite. The model runs locally; no external embedding service is used.

6.3 Retrieval Workflow

1. Receive the query from the agent's search_graph tool (or the search box).

2. Run a keyword search (BM25 ranking) and a vector search (cosine similarity, threshold 0.6).

3. Merge both lists with reciprocal rank fusion (score = Σ 1/(60 + rank)).

4. Return the top results, each labelled with how it matched: keyword, meaning or both.

5. The agent opens the most relevant entities and follows their relationships (graph-augmented retrieval).

6. Retrieved facts and documents are returned as tool results; the final answer cites the entities used.

6.4 Graph-Augmented Retrieval

Beyond similarity search, the follow_links tool walks typed relationships with a recursive SQL query up to four hops. This answers questions such as dependency blast radius that plain vector RAG cannot, because the answer is spread across several connected documents.

7. Tool/API Integration

7.1 Implemented Tools

Tool	Access	Function
search_graph	Read	Hybrid keyword + meaning search over the knowledge graph
open_entity	Read	Entity description, all facts with status and source, full document
follow_links	Read	Multi-hop traversal of relationships (1–4 hops)
graph_stats	Read	Number of entities, verified and proposed facts
review_queue	Read	Proposed facts waiting for human review
list_repos	Read · GitHub	Repositories the token can access
repo_files	Read · GitHub	File tree of a repository
read_file	Read · GitHub	Contents of one file (up to 16,000 characters)
search_code	Read · GitHub	Code search within a repository
propose_fact	Write · MCP only	Adds a fact as proposed (unverified)
verify_fact	Write · MCP only	Approves or rejects a proposed fact

7.2 GitHub REST API

The four GitHub tools call the GitHub REST API with a read-only personal access token stored in a local .env file. The agent chooses these tools on its own when a question is about real source code; for example, asked about a repository, it listed the files, read the README and requirements file and summarised the project.

7.3 Anthropic Messages API

The agent calls Claude Sonnet 5.5 through the Anthropic Messages API with the tool list, adaptive thinking (summarised reasoning shown to the user), medium effort, and server-side fallback for requests declined by safety classifiers.

7.4 Model Context Protocol (MCP) Server

mcp_server.py registers the same Python tool functions with the MCP Python SDK, so Claude Code, Claude Desktop or any MCP client can use the knowledge graph directly, over stdio or streamable HTTP (http://localhost:8031/mcp).

7.5 Source Code

Source code: https://github.com/VoyagerToko/agentic-ai-fa — three code files (app.py, mcp_server.py, static/index.html), the knowledge_base folder of documents, the report folder, requirements.txt and a README.

8. Agent Workflow

8.1 Step-by-Step Execution

1. The user asks a question; up to six previous turns are included for follow-up context.
2. The agent sends the question, system prompt and nine read-only tools to Claude.
3. Claude reasons (summary streamed to the user) and requests one or more tool calls.
4. The tool router executes them; entities touched are recorded and lit up on the graph.
5. All results are returned to Claude in one message; steps 3–5 repeat (maximum 25 rounds).
6. Claude returns the final Markdown answer, naming the entities used and flagging proposed facts.

8.2 Worked Example

Input: “What breaks if Postgres goes down?”

Observed trace: search_graph(“Postgres”) → open_entity(postgres) → search_graph(“dependsOn postgres”) → reasoning: “I should also check what else depends on auth-service before proceeding.” → search_graph(“depends on Auth Service or Billing Service”) → follow_links(web-app, 2) → follow_links(notification-service, 2) → reasoning → final answer. The answer listed the four services that depend directly on Postgres (Auth, Billing, Search, Analytics Pipeline), the indirect impact on the Web App and Mobile App, and noted uncertainty where the graph had no evidence. Twenty-two entities were highlighted on the graph in the order the agent reached them.

8.3 Planning and Re-Planning

The plan is not fixed in code: after each observation the model decides the next tool. In the example above it refined its search terms and added traversal steps when the first results were not enough. Tool errors are returned as text, so the agent can try a different tool or input instead of failing.

9. Safety, Ethics and Responsible AI

- Read-only agent: the agent receives only read tools; changes to data require a human (approval in the web page) or an explicit MCP client.
- Human-in-the-loop: uncertain facts are stored as proposed and must be approved before they count as verified.
- Provenance and honesty: every fact keeps its source document; answers flag proposed facts and are instructed never to invent facts.
- Model safety: refusals from Claude’s safety classifiers are handled; server-side fallback retries declined requests on a fallback model.
- Bounded autonomy: at most 25 reasoning rounds; tool outputs truncated to safe sizes.
- Secrets and privacy: API keys only in a local .env file, never committed; the GitHub token is read-only; the server listens on localhost only; demonstration data is fictional.
- Safe rendering: model answers are sanitised (DOMPurify) before display in the browser.

10. Results and Test Report

10.1 Test Cases

The following test cases were executed on the working application.

#	Test case	Input / action	Expected behaviour	Observed result	Status
1	Multi-step dependency analysis	"What breaks if Postgres goes down?"	Agent chains several tools and cites entities	8 steps (6 tool calls, 2 reasoning summaries); 4 direct and indirect impacts listed; 22 entities lit up	Pass
2	Ownership and standards	"Who owns the billing service and which standards must it follow?"	Correct owner and standards with source	Product Team (owner), Chloe Nguyen (maintainer), 3 standards; source file cited	Pass
3	Semantic retrieval	Search "payments"	Billing Service found	Billing Service ranked first (matched by keyword + meaning)	Pass
4	GitHub API tool	list_repos via MCP (HTTP)	Repositories returned	The token's repositories listed with language and description	Pass
5	Human review	Approve proposed fact "Mobile App mustSatisfy Testing Standard"	Fact becomes verified	Verified facts 73 → 74, proposed 2 → 1	Pass
6	MCP interoperability	MCP client over stdio: initialize, tools/list, follow_links(web-app)	All tools listed, call works	11 tools listed; dependency chain of Web App returned	Pass
7	Missing credentials	Run the agent with no API key	Clear error, no crash	Authentication error returned and shown as an error message	Pass
8	Read-only agent	Inspect the tool list sent to the model	Write tools not available to agent	Only the 9 read tools are sent; propose/verify available only via MCP or the web page	Pass

10.2 Discussion

All eight test cases passed. The tests show that the agent selects tools autonomously, combines several steps, retrieves by meaning as well as by keyword, and respects its safety boundaries. A stronger evaluation would measure answer accuracy and citation correctness on a labelled question set, retrieval precision, end-to-end completion rate and response latency.

11. Limitations

- The knowledge base is small and fictional (28 documents).
- Facts come from structured front matter; free-text relationships are not extracted automatically.
- A single agent handles all steps; there is no separate planner or verifier agent.

- Chat history is kept in memory and is lost when the server restarts.
- Requires an Anthropic API key and internet access for the model; no user authentication.

12. Future Scope

- Use the LLM to extract facts from unstructured documents and add them as proposed facts for review.
- Connectors for Google Drive, Confluence and issue trackers.
- A multi-agent design with planner, researcher and verifier roles.
- User authentication and role-based access to sensitive facts.
- An evaluation dashboard for accuracy, citation quality and latency.
- Docker deployment and persistent conversation storage.

13. Conclusion

Nexus demonstrates agentic behaviour in a practical setting: it plans which information it needs, retrieves it with hybrid RAG and graph traversal, calls external APIs, and produces grounded answers whose every step is visible. Separating verified from proposed facts and keeping the agent read-only shows how autonomy can be combined with human oversight. The same tools, exposed through MCP, make the knowledge base reusable by other AI assistants.

14. References and Tools Used

- Anthropic Claude API documentation — https://docs.claude.com
- Model Context Protocol — https://modelcontextprotocol.io
- Flask documentation — https://flask.palletsprojects.com
- SQLite FTS5 — https://www.sqlite.org/fts5.html
- fastembed — https://github.com/qdrant/fastembed
- BAAI/bge-small-en-v1.5 — https://huggingface.co/BAAI/bge-small-en-v1.5
- vis-network — https://visjs.github.io/vis-network/docs/network/
- GitHub REST API — https://docs.github.com/en/rest

Appendix A: How to Run

1. Clone https://github.com/VoyagerToko/agentic-ai-fa and open the folder in VS Code.

2. Create and activate a virtual environment: python -m venv .venv, then .venv\Scripts\activate

3. Install packages: pip install -r requirements.txt

4. Copy .env.example to .env and add ANTHROPIC_API_KEY and GITHUB_TOKEN.

5. Start the app: python app.py, then open http://localhost:8030

6. Optional, connect Claude Code via MCP: claude mcp add nexus -- .venv\Scripts\python.exe mcp_server.py

Appendix B: Project Structure

Path	Purpose
app.py	Server: graph, ingest, search, tools, agent loop and web routes
mcp_server.py	MCP server exposing the same tools
static/index.html	Web interface: Agent, Graph and MCP tabs
knowledge_base/	Company documents (services, teams, people, standards, tools, guides)
report/	Project report (Word, PDF, Markdown) and architecture diagram
data/	Generated SQLite database and embedding model cache
.env.example	Example environment settings
requirements.txt	Python package requirements
README.md	Setup and usage